

Ferrite Browser — Master Reference Index

What This Document Set Is

This is the complete, fool-proof reference for the Ferrite Browser project. Each section is self-contained — any section can be given to an AI tool or a new team member and they will have full context for that domain. The sections cross-reference each other where needed.

Project Identity

- **Name:** Ferrite Browser (working name)
- **Nature:** Developer-focused Rust browser with capability-based agent governance
- **License:** Dual MIT/Apache-2.0 (browser shell). Servo dependency is MPL-2.0.
- **Timeline:** 8 months, 3 people, ~24 person-months
- **Platform:** Linux-first (develop via WSL2 if on Windows). Docker devcontainer for build reproducibility. Windows port is a stretch goal for months 7-8.
- **Primary output:** Working browser prototype + two standalone developer tools + research paper

One-Paragraph Summary

Ferrite is a Rust browser built on the Servo engine that enforces capability-based governance over all extension and AI agent actions. Every privileged operation — DOM access, network requests, storage, clipboard, downloads — requires an unforgeable capability token issued by a central broker after policy evaluation. All actions are recorded in a tamper-evident audit log. This architectural approach complements model-level prompt injection defenses: even if an AI agent is successfully tricked by malicious webpage instructions, it physically cannot execute unauthorized actions because it lacks the capability tokens. The browser ships with a Wasm-sandboxed extension runtime, production-quality content blocking, accessibility tree access for agents, and polished developer inspection panels.

Section Index

#	Section	File	What It Covers
0	Master Index	<code>00-master-index.md</code>	This file. Project identity, section map, key decisions.
1	Objectives & Success Criteria	<code>01-objectives.md</code>	What we're building, for whom, what success looks like, what's explicitly out of scope.

#	Section	File	What It Covers
2	Architecture Overview	02-architecture.md	System-level architecture, process model, data flows, trust boundaries, module inventory.
3	Capability Broker & Token System	03-capability-broker.md	Token format, minting, validation, revocation, policy integration, enforcement semantics.
4	Policy Engine	04-policy-engine.md	Regorus/Rego integration, policy structure, risk classification, step-up consent, enterprise extensibility.
5	Extension Sandbox (Wasm/WASI)	05-extension-sandbox.md	Extism/Wasmtime architecture, host functions, capability injection, resource limits, extension manifest format.
6	Audit Log & Verification	06-audit-log.md	Event schema, hash chain, Merkle tree upgrade, storage design, proof generation/verification, encryption at rest.
7	Agent Governance	07-agent-governance.md	Agent protocol (JSON-RPC/WS), governance flow, risk classification, step-up consent, prompt injection defense model.
8	Threat Model & Security Analysis	08-threat-model.md	Attacker profiles, attack surfaces, trust boundaries, attack trees, mitigations, residual risks, known limitations.
9	UI & Developer Tools	09-ui-devtools.md	Iced shell design, custom panels, standalone tools, Firefox remote debugging integration, UX flows.
10	Engine Integration (Servo + CEF)	10-engine-integration.md	Servo embedding strategy, WebViewDelegate usage, CEF harness, engine abstraction layer, pinning strategy.
11	Network Stack & Content Blocking	11-network-blocking.md	adblock-rust integration, request interception, audit logging of blocks, HTTP/3 via quinn.
12	Accessibility Automation	12-accessibility.md	AX tree extraction, broker-gated access, agent integration, WCAG checks.
13	Implementation Plan	13-implementation-plan.md	Month-by-month roadmap, task dependencies, milestones, deliverables, critical path.

#	Section	File	What It Covers
14	Technology Stack & Dependencies	14-tech-stack.md	Every crate, version, license, maturity, integration notes, build system, compile time mitigations.
15	Evaluation & Measurement	15-evaluation.md	Benchmarks, adversarial test corpus, performance metrics, measurement methodology.

Key Decisions Already Made

These decisions are final and should not be re-debated in implementation:

1. **Servo pinning:** Pin to latest stable at project start. One planned upgrade at month 4 with a strict 1-week time-box — abort if it takes longer.
2. **Extension runtime:** Hybrid — Extism now, raw Wasmtime migration later.
3. **Capability scope:** 7+ capabilities in two waves. Wave 1 (months 2-3): dom.read, dom.write, network.fetch, storage.read, storage.write. Wave 2 (months 4-5): clipboard, notifications, downloads.
4. **Policy engine:** Regorus (pure-Rust OPA interpreter) from day one.
5. **UI framework:** Iced (pure Rust, wgpu-rendered, dark theme).
6. **Audit log:** Hash chain first, Merkle tree upgrade at month 5. Dual storage: append-only file (source of truth) + SQLite index (queries).
7. **DevTools:** Firefox remote debugging for standard panels + custom Iced panels for differentiating features.
8. **Agent protocol:** Custom JSON-RPC over WebSocket. Own demo agent first, Playwright adapter as stretch goal.
9. **Platform:** Linux-first. Docker devcontainer. WSL2 for Windows team members.
10. **Content blocking:** adblock-rust integration with EasyList + EasyPrivacy.
11. **Additional modules:** Agent governance + prompt-injection hardening, Accessibility automation.
12. **License:** Dual MIT/Apache-2.0 for browser shell.
13. **UI polish:** Polished developer tool aesthetic (clean, professional, dark mode).

Cross-Cutting Invariants

These rules apply across ALL sections and ALL implementation:

1. **No ambient authority.** No process, extension, or agent holds direct access to any privileged resource. Everything goes through the capability broker.
2. **Audit everything.** Every capability grant, denial, exercise, and revocation is logged. No silent failures.
3. **Fail closed.** If the broker is unreachable, policy evaluation fails, or a token is invalid, the action is DENIED. Never fail open.
4. **Trusted UI is separate.** Consent dialogs render in the Iced UI process, never in web content. An attacker controlling page content cannot spoof consent.
5. **Engine independence.** The capability broker, policy engine, audit log, and extension sandbox must not depend on Servo-specific APIs. They work with CEF (Track B) too.
6. **Deterministic serialization.** All audited events use canonical JSON serialization (sorted keys, no whitespace, UTF-8 normalized) so hash chains are reproducible.